

MRP-AI: RAG Q&A Bot

Mini Research Problem · Sofia University

MOTIVATION

Large Language Models (LLMs) confidently fabricate answers to questions they have no grounding for: a problem known as hallucination.

MAIN IDEA

Build a Retrieval-Augmented Generation (RAG) pipeline that grounds the LLM in an explicit document corpus. It answers from the documents — and refuses when the answer is not there.

KEY RESULT

RAG scored 20/20. Vanilla LLM scored 10/20.

Vanilla hallucinated a confident answer on 100% of out-of-scope questions. RAG correctly refused all 10.

Ketan Girish Hukare

Literature Review

Related work this MRP builds on

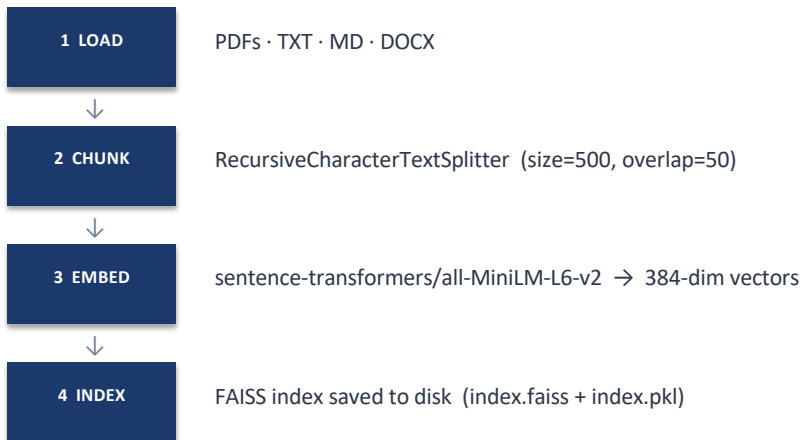
Paper / Resource	Authors & Year	Relevance to this MRP
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	Lewis et al., 2020	Foundational RAG architecture: combines dense retrieval with seq2seq generation
FAISS: A Library for Efficient Similarity Search	Johnson et al., 2017	Vector index used to store and retrieve document embeddings at scale
Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks	Reimers & Gurevych, 2019	Basis of the all-MiniLM-L6-v2 model used to embed chunks and queries
A Survey on Retrieval-Augmented Generation for LLMs	Gao et al., 2023	Taxonomy of RAG variants; motivates the compare-mode evaluation design
LangChain Framework	Chase, 2022–present	Orchestration layer connecting document loaders, splitters, embeddings, and LLMs

Approach

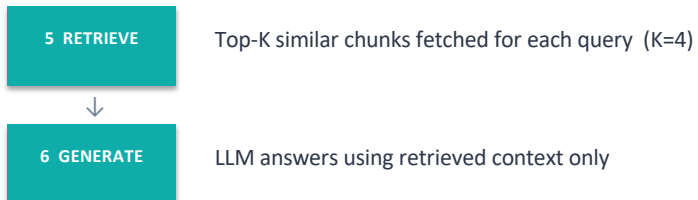
Two-phase pipeline: ingest once, query many times

PIPELINE

— Part 1: Ingestion —



— Part 2: Query —



TECH STACK

LangChain 0.3

Pipeline orchestration · python.langchain.com

FAISS

Local vector similarity search · faiss.ai

all-MiniLM-L6-v2

Sentence embedding model (~90 MB) · huggingface.co

Ollama / llama3.2

Local LLM, no API key needed · ollama.com

sentence-transformers

Embedding library · sbert.net

Code

Core implementation — Part 1 (ingest) and Part 2 (RAG chain)

PART 1 — BUILD INDEX (PART1_INGEST.PY)

```
# Split documents into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=cfg.CHUNK_SIZE,
    chunk_overlap=cfg.CHUNK_OVERLAP
)
chunks = splitter.split_documents(docs)

# Embed and save to FAISS
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
vectorstore = FAISS.from_documents(chunks, embeddings)
vectorstore.save_local(cfg.INDEX_SAVE_PATH)
```

PART 2 — RAG CHAIN (PART2_RAG.PY)

```
# Load index and build retriever
vectorstore = FAISS.load_local(
    cfg.INDEX_SAVE_PATH, embeddings,
    allow_dangerous_deserialization=True
)
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": cfg.RETRIEVAL_TOP_K}
)

# Build RAG chain
chain = RetrievalQA.from_chain_type(
    llm=llm, chain_type='stuff',
    retriever=retriever,
    return_source_documents=True,
    chain_type_kwargs={'prompt': RAG_PROMPT}
)
```

Experiments & Results

8 RAG configurations + vanilla LLM baseline · 20 questions · 200 total queries

Experiment	Chunk	Overlap	Top-K	Total	Answerable	Unanswerable
baseline	500	50	4	20/20	10/10	10/10
chunk_200	200	50	4	20/20	10/10	10/10
chunk_1000	1000	50	4	20/20	10/10	10/10
overlap_0	500	0	4	20/20	10/10	10/10
overlap_100	500	100	4	20/20	10/10	10/10
topk_1	500	50	1	19/20	9/10	10/10
topk_2	500	50	2	19/20	9/10	10/10
topk_8	500	50	8	20/20	10/10	10/10
vanilla_llm	—	—	—	10/20	10/10	0/10

KEY FINDINGS

RAG vs Vanilla LLM

RAG 20/20 · Vanilla 10/20.
Vanilla hallucinated on every out-of-scope question (0/10). RAG refused all 10 correctly.

Top-K is most sensitive

K=1 and K=2 each missed 1 answer. K=4 is the sweet spot — multi-part questions need multiple chunks.

Chunk size & overlap

No accuracy difference across all tested values on this corpus. Differences surface on larger, noisier datasets.

Recommended config

chunk_size=500 · overlap=50 · top_k=4

Live Demo



YouTube Demo

<https://youtu.be/hNoRVCKN7r0>

Demonstrates: document ingestion · RAG Q&A · --compare mode (RAG vs Vanilla side-by-side)

DEMO COVERS

- Run `part1_ingest.py` — scan 3 documents, build FAISS index
- Run `part2_rag.py` — ask questions answered by the corpus
- Run `part2_rag.py --compare` — same question, RAG vs vanilla LLM, side-by-side
- Show a hallucination example from the vanilla LLM vs correct refusal from RAG

Model: llama3.2 via Ollama · Runs fully locally, no internet required